EventOps AI

Reliability and Evaluation Report

SE4471B Course Project | Assignment 3

Sara Daher · Aya Maree

April 2026

---

1. System Overview

EventOps AI is a conversational AI event planning system that transforms unstructured natural-language event requirements into a complete, conflict-checked planning package. The final system achieves Tier 2 (Tool-Enabled System), incorporating:

- **RAG pipeline:** sentence-transformers (all-MiniLM-L6-v2) + ChromaDB over a 23-document knowledge base
- **Planning workflow:** 7-step agentic state machine with conditional logic
- **Memory:** In-memory session state (EventContext, ChatHistory, PlanningSession) across all turns
- **LLM:** Google Gemini 2.0 Flash, with a comprehensive template-based fallback (demo mode)
- **External API:** Spoonacular Food API for recipe-based shopping list enrichment (Tier 2)
- **Frontend:** React + Vite + Tailwind CSS with live workflow progress tracking

---

2. Testing Strategy

2.1 Manual End-to-End Testing

The primary approach was scenario-based end-to-end testing — running complete planning workflows and verifying the system's behavior at each step.

Scenario	Expected Behavior	Outcome
Complete intake in one message	System extracts all 5 fields, skips to retrieval	Pass
Intake across multiple turns	Context accumulates incrementally across messages	Pass
Budget conflict: < $8/person	Conflict detection fires, halts planning, asks for resolution	Pass
Tight timeline: event < 7 days away	Timeline conflict flagged with specific actionable warning	Pass
Vegetarian + gluten-free guests	dietary_restrictions extracted, retrieves dietary_guidelines.json	Pass

Children attending	has_children = true, retrieves entertainment/activities content	Pass
Artifact generation	All 3 artifacts produced (checklist, shopping list, schedule)	Pass
Recipe suggestions (Spoonacular on)	Shopping list enriched with real recipe ingredients	Pass
Recipe suggestions (Spoonacular off)	Fallback recipes with pre-defined ingredient lists used instead	Pass
Post-artifact adjustments	User can modify context and regenerate after plan is complete	Pass

2.2 Context Extraction Testing

Context extraction was tested with various natural-language phrasings to verify that the regex patterns and Gemini-based parsing handled them correctly:

Input Phrase	Field Tested	Result
"20 guests"	guest_count_estimated	Correct
"guests to 15"	guest_count_estimated	Correct (after regex fix)
"$300"	budget_total	Correct
"budget to 400"	budget_total	Correct (after regex fix)
"2026-04-25"	event_date (ISO)	Correct
"April 25"	event_date (written)	Correct (after date parser added)
"at home"	venue_type	Correct
"birthday party"	event_type	Correct
"2 vegetarians"	dietary_restrictions	Correct

2.3 RAG Retrieval Quality Testing

Retrieval was tested by issuing targeted queries and verifying that semantically relevant documents ranked highest:

Query	Top Retrieved Document	Relevance Score
birthday party planning	birthday_party_guide.json	0.81
dietary vegetarian gluten-free	dietary_guidelines.json	0.78
shopping 20 guests food quantities	shopping_list_templates.json	0.75
day of schedule timeline setup	day_of_schedule_samples.json	0.77
children entertainment activities	entertainment_ideas.json	0.72
outdoor venue accessibility	accessibility_guide.json	0.68

2.4 Fallback Mode (Demo Mode) Testing

The system was deliberately tested without a valid GOOGLE_API_KEY to verify that:

- Template-based responses are coherent and collect missing fields correctly
- Context extraction using regex (not LLM) correctly parses all required fields
- The workflow still advances through all 7 steps to artifact generation
- Artifacts are generated using artifact_generator.py templates

Fallback mode produces a functional — though less nuanced — planning experience, which is by design.

2.5 API Failure Testing

Failure Injected	System Behavior
No GOOGLE_API_KEY	Falls back to template responses; workflow still completes
Invalid / expired Gemini key (404)	Falls back to template responses
Gemini quota exceeded (429)	Falls back to template responses
No SPOONACULAR_API_KEY	Uses _fallback_recipes with pre-defined ingredient lists
Spoonacular search returns 0 results	Uses fallback; randomized subset prevents repetition
Backend restart mid-session	Session lost (in-memory); user must start a new session

---

3. Failure Cases and Limitations

3.1 Known Failure Cases

F1 - Session Loss on Backend Restart

Description: All session data (event context, chat history, workflow state, artifacts) is stored in Python process memory. If the backend server restarts or crashes, all active sessions are lost.

Impact: Users must start a new planning session from scratch.

Mitigation: PlanningSession.to_dict() / from_dict() serialization exists and SessionManager accepts a persist_dir parameter, but file-based persistence was not activated in this implementation. Enabling it would resolve this limitation.

F2 - Context Extraction Failures in Complex Phrasing

Description: The fallback regex-based extractor handles common phrasings but can miss unusual constructions (e.g., "we'll have roughly two dozen people", or implied budgets like "maybe two fifty").

Impact: In demo/fallback mode, the user may be asked to re-enter information already provided.

Mitigation: When Gemini is available, the LLM handles these gracefully. Regex patterns were iteratively expanded to cover the most common phrasings found in testing.

F3 - Conflict Detection Loops

Description: If a detected conflict cannot be resolved (e.g., a date that is already past), the system can get stuck asking the user to resolve it repeatedly.

Impact: The planning workflow stalls at the clarification step.

Mitigation: A bypass was added: if ctx.detected_conflicts is already set when the user sends a message, the system routes directly to planning and clears the conflict list, allowing progress even without a resolution.

F4 - Spoonacular API Zero-Result Queries

Description: Overly specific search queries (e.g., "birthday party vegetarian finger food for 20 guests") frequently returned 0 results from the Spoonacular API.

Impact: The enrichment step silently fails; shopping list contains no recipe suggestions.

Mitigation: Search queries were simplified to core food terms. Random offsets were added for variety. _fallback_recipes with complete ingredient lists ensures suggestions always appear.

F5 - Artifact Over/Undergeneration

Description: The LLM may occasionally produce minimal (too few tasks/items) or verbose artifacts depending on how event context was described.

Impact: Artifacts may not reflect the full scope of the event if context is sparse.

Mitigation: artifact_generator.py provides template-based fallback artifacts calibrated to event type, guest count, and budget to ensure a reasonable baseline.

F6 - No Cross-Session Knowledge

Description: Each planning session is completely isolated. The system has no memory of previous users, prior events, or commonly requested event types.

Impact: No personalization or learning from past sessions.

Mitigation: By design for this implementation. Addressable with a user account system and persistent logs.

3.2 System Limitations Summary

Limitation	Notes
In-memory only	Sessions lost on server restart; persist_dir option not activated
Single concurrent user per session	Sessions are isolated; no collaborative planning
No real-time conflict checking	Conflicts checked only at conflict_detection step
No calendar integration	Dates tracked as strings; no iCal / Google Calendar sync
Gemini rate limits	Free tier ~1,500 req/day; heavy use triggers 429 → fallback mode
English only	Regex patterns and prompts are English-specific
Static knowledge base	23 documents authored at build time; no live web updates

---

4. Prompt and Workflow Design Decisions

4.1 System Prompt Design

The base system prompt (SYSTEM_PROMPT_BASE in llm_service.py) was built around four constraints:

- **Force KB grounding:** The prompt instructs: "Base ALL recommendations on the provided knowledge base context" and "Never fabricate costs or quantities; use the retrieved documents as your reference." Without this, Gemini tends to produce confident but hallucinated numbers that contradict the knowledge base.
- **Explicit citation requirement:** "Cite your sources explicitly using the format [Source: document_name]." Citations serve two purposes: they make the system's reasoning auditable, and they provide the frontend with structured metadata to display alongside responses.
- **Conflict detection as first-class behavior:** "Detect and flag conflicts (e.g., budget too low for guest count, timeline too tight)." By framing conflict detection as a core behavior, the LLM surfaces budget/timeline/dietary conflicts consistently without needing a separate prompt for each scenario type.
- **Context injection at every call:** Each LLM call includes the current EventContext as a JSON summary and the current workflow state. The model always has the full planning state, not just the current message; essential for multi-turn coherence.

4.2 Workflow State Machine Design

The 7-step workflow is a conditional state machine rather than a fixed linear sequence. Key design decisions:

- **Why not a simple chain-of-thought?:** A single "plan this event" prompt has no mechanism to handle missing information, detect conflicts before generating artifacts, or update the plan when context changes. The state machine ensures each concern is handled in the correct order.
- **Intake → Auto-advance:** When all 5 required fields are collected, the system automatically advances without requiring an explicit user command. This avoids friction and keeps the conversation flowing naturally.
- **RAG during intake (not just retrieval step):** The system retrieves KB chunks even during the intake step so early responses are grounded; e.g., if the user says "birthday party" but hasn't provided a budget yet, responses still reference birthday_party_guide.json.
- **Conflict detection gate:** Artifacts cannot be generated until conflicts are resolved (or overridden). This guardrail prevents producing a shopping list for a $5/person budget without flagging the issue.
- **Validation as a free-chat zone:** Between planning and artifact generation, the validation step allows open-ended questions and adjustments. It triggers a fresh RAG retrieval for each question, so answers remain grounded.
- **Adjustment keyword detection:** In validation, adjustment-intent keywords trigger a re-run of the full planning pipeline rather than just a chat reply. This handles cases like "change the budget to $500" — the system updates EventContext, re-retrieves, and re-generates the plan.

4.3 Context Accumulation Design

The EventContext object accumulates across turns rather than resetting. Every user message is processed through extract_event_context_from_intake() before routing to the workflow step

handler. This means a user who provides information across three separate messages gets the same result as one who provides everything at once.

The get_summary() method on EventContext serializes the current state into a compact string injected into every LLM call, ensuring the model always has the full planning context.

4.4 Fallback Design Philosophy

The fallback (demo mode) was treated as a distinct, functional mode rather than a degraded experience:

- Dynamic responses: The fallback acknowledges already-collected fields and lists only remaining missing ones; not a static boilerplate.
- Regex breadth: Multiple patterns cover the most common phrasings for each field, including edge cases from testing.
- Template artifacts: artifact_generator.py generates structured artifacts calibrated to event type, guest count, and budget.

---

5. Evidence of Retrieval Grounding

5.1 How Retrieval Grounding Works

At every workflow step, the system retrieves 4–8 semantically relevant chunks from the knowledge base using the sentence-transformers model and ChromaDB vector store. These chunks are formatted into a context block injected into the LLM prompt:

```
[Source 1: Birthday Party Planning Guide | doc_id: birthday_party_guide |
similarity: 81%]
<chunk text>

[Source 2: Event Budget Planning Guide | doc_id: budget_planning_guide |
similarity: 74%]
<chunk text>
```

5.2 Query Enrichment

The retrieval query is enriched with the current event context before embedding, improving relevance. For example, a query of "shopping list" for a vegetarian birthday party with 20 guests becomes:

```
enriched_query = "shopping list birthday party home dietary vegetarian 20
guests"
```

This yields higher relevance scores for shopping_list_templates.json and dietary_guidelines.json than the bare query would.

5.3 Citations Visible to the User

Every response carries a citations array displayed in the UI alongside the AI message, making retrieval grounding transparent. During testing, citations consistently matched the query context:

Workflow Step	Documents Retrieved
Intake	Birthday Party Planning Guide, Event Budget Planning Guide
Retrieval	Birthday Party Planning Guide, Shopping List Templates, Dietary Guidelines, Budget Planning Guide, Day-of Schedule Samples
Conflict Detection	Event Budget Planning Guide (flagged: $15/person low for sit-down meal)
Planning	Birthday Party Planning Guide, Catering Guidelines, Vendor & Decoration Ideas
Artifact Generation	Shopping List Templates, Day-of Schedule Samples, Catering Guidelines

5.4 Conflict Detection Grounded in KB

Conflict detection is not rule-based — the LLM receives the retrieved KB context and is asked to identify issues against knowledge-base guidelines. This means conflict messages are sourced from the knowledge base, not from hardcoded thresholds. For example:

- "Your budget of $300 for 20 guests is $15/person. The knowledge base recommends $20–35/person for a home birthday party with a sit-down meal."
- "The event is 12 days away. Custom bakery cakes typically require 1–2 weeks lead time."

The system adapts to different event types because the retrieved chunks differ, what constitutes a budget conflict for a formal dinner party differs from a casual birthday.

5.5 Spoonacular as a Second Grounding Layer

For shopping list enrichment, the Spoonacular Food API provides a second grounding layer beyond the static knowledge base:

- The system identifies the event type from EventContext
- A targeted recipe search query is constructed (e.g., "birthday cake dessert")
- Spoonacular returns real recipe data with ingredient quantities scaled to the guest count
- Ingredients are merged into the shopping list with estimated costs

5.6 Sample Retrieval → Response Chain

For a vegetarian birthday party with 20 guests, $300, the system retrieved the following from dietary_guidelines.json:

```
"For vegetarian guests, ensure at least 30% of food options are vegetarian-
friendly.
 Label all items clearly at the buffet. Consider vegetarian protein sources:
hummus,
 cheese platters, stuffed mushrooms, vegetarian pasta."
```

The resulting planning response included: "Since you have vegetarian guests, the knowledge base recommends dedicating at least 30% of your food options to vegetarian choices, hummus and veggie platters, stuffed mushrooms, and vegetarian pasta work well for 20 guests at this budget [Source: Dietary Guidelines]."

This demonstrates the full chain: RAG retrieval → context injection → grounded generation → cited response.

Appendix — System Architecture Summary

```
User Message
    ↓
extract_event_context_from_intake()     ← Gemini JSON extraction + regex
fallback
    ↓
EventContext updated
    ↓
Route to workflow step handler
    ↓
RAG retrieve (sentence-transformers + ChromaDB)
    enriched_query = user_message + event context fields
    top-k = 4 to 8 chunks
    ↓
_build_rag_context() → context string for LLM prompt
    ↓
chat_with_context() → Gemini API call  (or template fallback)
    system_prompt: SYSTEM_PROMPT_BASE + event context + RAG context
    messages: last 6-10 turns of chat history
    ↓
Response + citations returned to frontend
    ↓
(At artifact_generation step):
generate_artifact_json() × 3 → Task Checklist, Shopping List, Day-of Schedule
    ↓
(If Spoonacular enabled):
enrich_shopping_list() → recipe search → ingredient scaling → merged list
```